

OSSP WEEK 5 SKILL

Topic: To Implement Single Quote Handling for Literal String Preservation

1. Created a folder for week5 skill

```
samanth-ubuntu@samanth-ubuntu:~$ mkdir week5_s
samanth-ubuntu@samanth-ubuntu:~$ cd week5_s
samanth-ubuntu@samanth-ubuntu:~/week5_s$ pwd
/home/samanth-ubuntu/week5_s
samanth-ubuntu@samanth-ubuntu:~/week5_s$ |
```

2. Single quote vs Double quote check

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ name="Samanth"
samanth-ubuntu@samanth-ubuntu:~/week5_s$
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo 'Hello $name'
Hello $name
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Hello $name"
Hello Samanth
```

3. Single quotes preserve literal content

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo '$HOME'
/home/samanth-ubuntu
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo '$USER'
samanth-ubuntu
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo 'Today is $(date)'
Today is $(date)
```

4. Store quoted strings in variables

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ msg="Week 5 Shell Assignment"
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo $msg
Week 5 Shell Assignment
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Welcome to $msg!"
Welcome to Week 5 Shell Assignment!
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo 'Message:'
Message:
```

5. Demonstrate spaces with double quotes

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ text="Linux Shell Programming"
samanth-ubuntu@samanth-ubuntu:~/week5_s$
samanth-ubuntu@samanth-ubuntu:~/week5_s$ printf '<%s>\n' "$text"
<Linux Shell Programming>
samanth-ubuntu@samanth-ubuntu:~/week5_s$ printf '<%s>\n' $text
<Linux>
<Shell>
<Programming>
```

6. Argument Parsing

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ show_args() {
> echo "Number of arguments: $#"
```

```
> for arg in "$@"
> do
>     echo "Argument: [$arg]"
> done
>
>
> c
> ^C
samanth-ubuntu@samanth-ubuntu:~/week5_s$ show_args() {
> echo "Number of arguments: $#"
```

```
> for arg in "$@"; do
>     echo "Argument: [$arg]"
> done
> }
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ show_args "Linux Shell Programming"
```

```
Number of arguments: 1
Argument: [Linux Shell Programming]
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ show_args Linux Shell Programming
```

```
Number of arguments: 3
Argument: [Linux]
Argument: [Shell]
Argument: [Programming]
```

7. Nested Tokens / Command Substitution

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Current user: $USER"
```

```
Current user: samanth-ubuntu
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Current directory: $(pwd)"
```

```
Current directory: /home/samanth-ubuntu/week5_s
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Current date: $(date)"
```

```
Current date: Thu Sep 17 16:42:08 UTC 2026
```

8. Quoted Command

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ name="Samanth"
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo 'Hello $name'
```

```
Hello $name
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ echo "Hello $name"
```

```
Hello Samanth
```

9. Parsing Validation

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ nano valid.sh
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ bash -n valid.sh
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ bash valid.sh
```

```
Hello Samanth
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ nano invalid.sh
```

```
samanth-ubuntu@samanth-ubuntu:~/week5_s$ nano invalid.sh
```

```
samanth@DESKTOP-BLT5L3R:~/week5_s$ bash -n invalid.sh
invalid.sh: line 2: unexpected EOF while looking for matching `"'
```

1. Final Combined Implementation

```
samanth@DESKTOP-BLT5L3R:~/week5_s$ nano week5_quotes.sh
samanth@DESKTOP-BLT5L3R:~/week5_s$ bash -n week5_quotes.sh
samanth@DESKTOP-BLT5L3R:~/week5_s$ chmod +x week5_quotes.sh
samanth@DESKTOP-BLT5L3R:~/week5_s$ ./week5_quotes.sh
Single quotes: $name
Double quotes: Samanth
$HOME
$USER
<Linux Shell Programming>
<Linux>
<Shell>
<Programming>
Current user: samanth
Current directory: /home/samanth/week5_s
echo "Hello from a quoted command"
Hello from a quoted command
It's Linux
```

Summary:

In this practical, we learned how Bash handles single and double quotes while executing shell commands. We demonstrated literal string preservation, variable expansion, spaces in strings, argument parsing, and command substitution using commands such as echo, printf, and `$(...)`. We also practiced quoted commands and syntax validation using `bash -n`. Finally, we created and executed a combined Bash script to verify all the concepts practically and checked the results through terminal outputs and screenshots.